

POLITECHNIKA POZNAŃSKA
WYDZIAŁ AUTOMATYKI, ROBOTYKI I
ELEKTROTECHNIKI

Instytut Robotyki i Inteligencji Maszynowej
Zakład Automatyki i Optymalizacji



Jan Andrzejewski
Mateusz Banaszak
Piotr Bednarek

PROJEKT I BUDOWA UKŁADU REGULACJI
DLA UKŁADU BALL ON BEAM

Promotor: dr inż. Jacek Michalski

Pracę przyjmuje i akceptuje

.....
data i podpis opiekuna pracy

Poznań, 2026

Streszczenie

Praca przedstawia projekt i budowę autonomicznego stanowiska do sterowania pozycją kulki na belce (ball and beam) z wykorzystaniem mikrokontrolera STM32F767ZI. Opisano budowę mechaniczną stanowiska, zastosowane peryferia (serwomechanek MG946R, kamera systemu wizyjnego, potencjometr) oraz architekturę oprogramowania opartą na systemie FreeRTOS. Wyprowadzono model matematyczny układu i zaprojektowano dwa regulatory: klasyczny PID z anti-windup oraz optymalny LQR w przestrzeni stanów. Aplikacja nadzorcza napisana w Pythonie (PySide6) umożliwia wizualizację sygnałów w czasie rzeczywistym, zapis danych do pliku CSV oraz zdalne strojenie regulatora. Przeprowadzono weryfikację eksperymentalną i potwierdzono zgodność zachowania fizycznego systemu z wyznaczonym modelem.

Słowa kluczowe: *regulacja automatyczna, kulka na belce, STM32, PID, LQR, FreeRTOS, systemy wizyjne.*

Abstract

Title: *Design and Construction of a Control System for the Ball and Beam System.*

This thesis presents the design and construction of an autonomous ball-and-beam control system based on the STM32F767ZI microcontroller. The mechanical setup, hardware peripherals (MG946R servo, computer vision camera, potentiometer), and FreeRTOS-based software architecture are described. A mathematical model of the plant is derived and two controllers are designed: a classical PID with anti-windup and an optimal LQR state-feedback controller. A supervisory desktop application (Python/PySide6) provides real-time signal visualisation, CSV data logging, and remote parameter tuning. Experimental verification confirms that physical system behaves the same as the theoretical model.

Key words: *automatic control, ball and beam, STM32, PID, LQR, FreeRTOS, computer vision.*

Spis treści

1	Wprowadzenie	3
1.1	Cel i zakres pracy	3
1.2	Przegląd literatury	4
1.3	Struktura pracy	4
2	Budowa stanowiska i oprogramowanie	5
2.1	Opis stanowiska laboratoryjnego	5
2.1.1	Wykaz komponentów	5
2.1.2	Architektura systemu i przepływ sygnałów	5
2.2	Mikrokontroler STM32 i peryferia	7
2.3	Środowisko programistyczne	9
2.3.1	Firmware STM32	9
2.3.2	Aplikacja desktopowa	10
2.4	Implementacja algorytmu sterowania	11
2.4.1	Pętla sterowania i okres próbkowania	11
2.4.2	Ograniczenie zakresu serwa	12
2.4.3	Protokół telemetry UART z sumą kontrolną CRC-8	12
2.4.4	Zadania FreeRTOS	12
2.5	System wizyjny	13
2.5.1	Konfiguracja kamery i akwizycja obrazu	13
2.5.2	Detekcja kulki metodą progowania w przestrzeni HSV	13
2.5.3	Wyznaczanie pozycji belki na podstawie znaczników ArUco	15
2.5.4	Kalibracja geometryczna i przeliczanie piksel $\rightarrow$ mm	16
2.5.5	Protokół komunikacyjny PC $\rightarrow$ STM32	17
3	Model matematyczny i projekt regulacji	18
3.1	Model matematyczny obiektu regulacji	18
3.1.1	Dynamika serwomechanizmu	18
3.1.2	Dynamika kulki na belce	19
3.1.3	Połączenie mechaniczne i transmitancja całkowita	20
3.2	Identyfikacja parametrów modelu	21
3.2.1	Identyfikacja wzmocnienia metodą odpowiedzi skokowej	21
3.2.2	Porównanie wzmocnienia teoretycznego i doświadczalnego	29
3.2.3	Przyjęte parametry modelu	31
3.3	Projekt regulatora LQR	31

3.3.1	Opis przestrzeni stanów	31
3.3.2	Sterowalność	31
3.3.3	Obserwowalność	32
3.3.4	Regulator LQR	32
3.4	Projekt regulatora PID	32
4	Wyniki eksperymentalne	34
4.1	Metodyka przeprowadzonych badań	34
4.2	Wyniki pomiarów	34
4.3	Analiza i omówienie wyników	34

Wprowadzenie

Układ kulka na belce (*ball and beam*) jest klasycznym laboratoryjnym obiektem sterowania, powszechnie stosowanym w dydaktyce automatyki ze względu na prostą budowę mechaniczną i zarazem nieliniową, niestabilną dynamikę [1]. Zadanie sterowania polega na utrzymaniu toczącego się swobodnie po belce metalowego walca w zadanej pozycji poprzez regulację kąta wychylenia belki napędzanej serwomechanizmem. Pomimo pozornej prostoty obiekt ten wymaga zastosowania metod nowoczesnej teorii sterowania, co czyni go idealną platformą do weryfikacji algorytmów regulacji zarówno klasycznych, jak i optymalnych.

1.1 Cel i zakres pracy

Celem niniejszej pracy jest zaprojektowanie i zbudowanie autonomicznego stanowiska sterowania kulką na belce opartego na mikrokontrolerze STM32F767ZI [2] oraz zaimplementowanie i porównanie dwóch algorytmów regulacji:

- regulatora **PID** (*Proportional-Integral-Derivative Regulator*) z filtrem pochodnej, członem anti-windup i strojeniem w oparciu o odpowiedź skokową i sygnał APRBS,
- regulatora **LQR** (*Linear Quadratic Regulator*) w przestrzeni stanów z wektorem stanu obejmującym pozycję kulki, jej prędkość oraz kąt belki.

Zakres pracy obejmuje:

- (i) budowę stanowiska laboratoryjnego z systemem wizyjnym [3] i serwomechanizmem MG90S,
- (ii) identyfikację modelu matematycznego obiektu metodami doświadczalnymi,
- (iii) projektowanie i dobór nastaw regulatorów PID oraz LQR,
- (iv) implementację algorytmów w środowisku FreeRTOS na platformie STM32,
- (v) opracowanie aplikacji nadzorczej w języku Python (PySide6) z podglądem kamery i detekcją pozycji kulki metodami wizji komputerowej [4],
- (vi) eksperymentalną weryfikację i porównanie obu strategii sterowania.

1.2 Przegląd literatury

Układ kulka na belce jest od dziesięcioleci obiektem intensywnych badań w dziedzinie automatyki i robotyki. Pierwszym krokiem projektowania regulatora jest zawsze wyrowadzenie modelu matematycznego. Bolívar-Vincenty i Beauchamp-Báez [1] wykazali, że równania ruchu otrzymane metodą Newtona i metodą Lagrange’a są tożsame, o ile poprawnie uwzględni się człony wynikające z obrotu układu odniesienia — błąd często popełniany w literaturze. Autorzy wyprowadzają nieliniowe równania stanu, a następnie linearyzują je w otoczeniu punktu równowagi, uzyskując transmitancję i liniowy model przestrzeni stanów, stanowiący punkt wyjścia do syntezy regulatora.

Wśród klasycznych podejść do sterowania układem dominuje regulator PID. Taifour Ali i in. [5] przedstawili implementację dwupętłowego regulatora PID na mikrokontrolerze Arduino z ultradźwiękowym czujnikiem odległości i serwomechanizmem DC. Autorzy potwierdzili skuteczność ręcznego doboru nastaw metodą prób i błędów.

Projektowanie optymalnych regulatorów stanu wymaga solidnych podstaw teoretycznych. Ogata [6] oraz Franklin i in. [7] dostarczają kompletnego opisu teorii przestrzeni stanów, analizy sterowalności i obserwowalności oraz syntezy LQR minimalizującego kwadratowy wskaźnik jakości. W niniejszej pracy macierze wag $\mathbf{Q}$ i $\mathbf{R}$ dobrano metodą iteracyjną z pomocą środowiska MATLAB/Simulink, a uzyskane wzmocnienia $\mathbf{K}$ przeniesiono bezpośrednio do firmware’u mikrokontrolera.

Dodatkową funkcją stanowiska jest wizualna weryfikacja pozycji kulki za pomocą kamery. Przegląd metod śledzenia obiektów w wizji komputerowej przeprowadzony przez Kadama i in. [4] wskazuje, że klasyczne podejście oparte na transformacji przestrzeni barw HSV i detekcji okręgów metodą Hougha pozostaje skuteczne dla obiektów o regularnym kształcie i kontrolowanym oświetleniu — warunki spełnione na stanowisku laboratoryjnym. Głębsze metody uczenia maszynowego byłyby uzasadnione w środowiskach o zmiennym tle lub przy wielu obiektach; dla niniejszego zastosowania przyjęto rozwiązanie klasyczne.

1.3 Struktura pracy

Praca podzielona jest na pięć rozdziałów. Rozdział 2 opisuje budowę mechaniczną i elektryczną stanowiska oraz architekturę oprogramowania — firmware FreeRTOS i aplikację nadzorczą Python. Rozdział 3 zawiera wyprowadzenie modelu matematycznego obiektu metodą Lagrange’a, linearyzację w punkcie pracy oraz projekt regulatorów PID i LQR. Rozdział 4 prezentuje wyniki eksperymentów identyfikacyjnych i regulacyjnych, w tym porównanie charakterystyk skokowych obu strategii sterowania. Rozdział ?? podsumowuje pracę i wskazuje kierunki dalszych badań.

Budowa stanowiska i oprogramowanie

Niniejszy rozdział opisuje fizyczną budowę stanowiska laboratoryjnego, zastosowane komponenty elektroniczne, środowisko programistyczne oraz architekturę oprogramowania sterującego — firmware mikrokontrolera i aplikację nadzorczą.

2.1 Opis stanowiska laboratoryjnego

Stanowisko składa się z wydrukowanej w technologii 3D pochyłej belki, po której swobodnie toczy się kulka (pileczka ping pongowa) –rys. 2.1. Kąt nachylenia belki kontrolowany jest przez serwomechanek połączony z belką za pomocą dźwigni. Pozycja kulki mierzona jest bezdotykowo za pomocą zaimplementowanego systemu wizyjnego. Mikrokontroler STM32 odczytuje pomiar z czujnika, oblicza sygnał sterujący i generuje odpowiedni sygnał PWM dla serwomechanizmu.

2.1.1 Wykaz komponentów

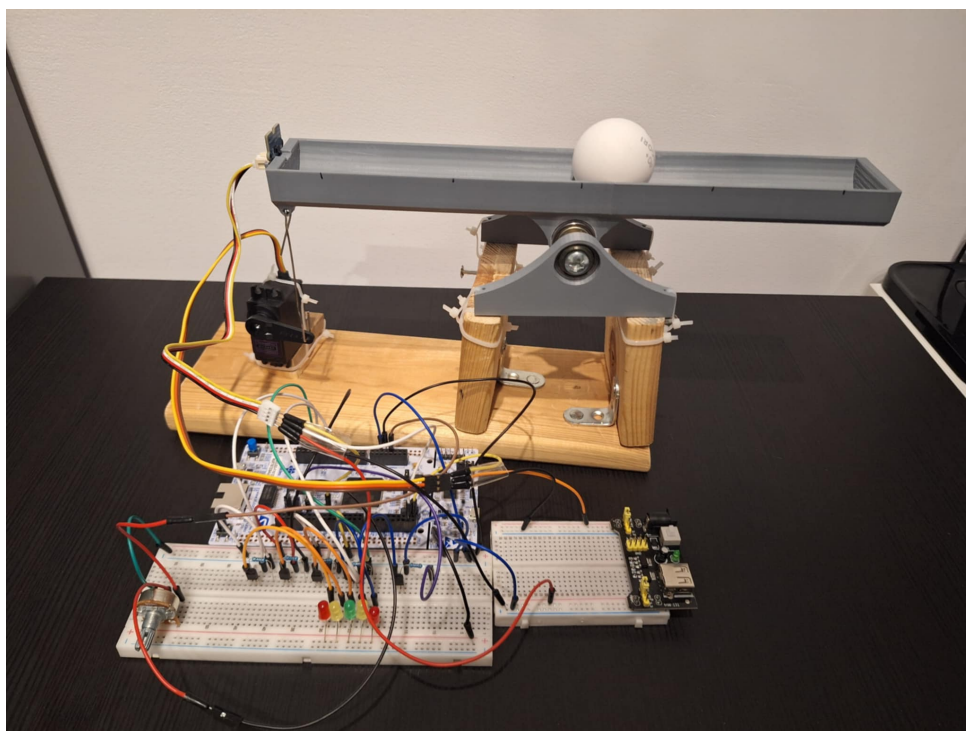
Komponenty wykorzystane przy budowie stanowiska zestawiono w tabeli 2.1.

Tabela 2.1: Komponenty sprzętowe stanowiska

Element	Model / opis	Interfejs
Mikrokontroler	STM32 Nucleo F767ZI	—
Servomechanek	TowerPro MG946R	PWM 50 Hz
Kamera USB	rozdzielczość 640×480, 30 FPS	USB
Potencjometr	liniowy	ADC (PA0)
Zasilacz	5 V DC	—

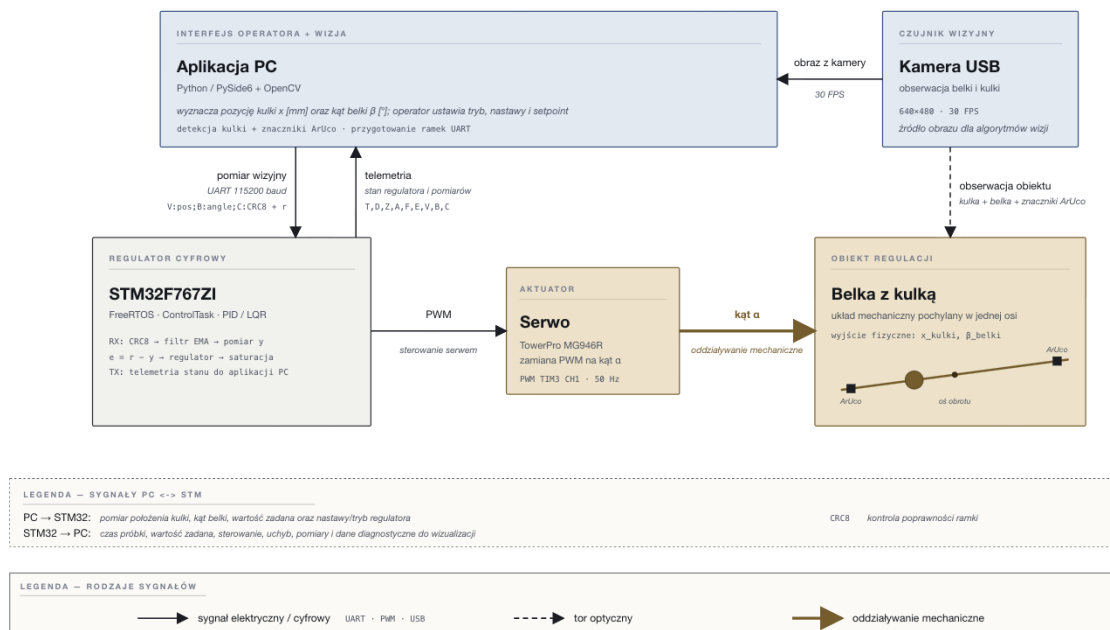
2.1.2 Architektura systemu i przepływ sygnałów

Powiązania między poszczególnymi podsystemami stanowiska oraz przepływ sygnałów pomiarowych, sterujących i telemetrycznych przedstawiono na rys. 2.2. Pętla regulacji zamknięta jest przez tor wizyjny: kamera USB obserwuje belkę i kulkę, aplikacja desktopowa



Rysunek 2.1: Stanowisko laboratoryjne układu kulka na belce

wyznacza pozycję kulki oraz kąt belki, a następnie przekazuje te wielkości do mikrokontrolera STM32 poprzez interfejs UART (115200 baud, ramka `V:<pos>;B:<angle>;C:<CRC>`). Mikrokontroler realizuje algorytm regulacji (PID lub LQR) i steruje serwomechanizmem sygnałem PWM, który zmienia kąt belki – zamykając pętlę poprzez fizyczne oddziaływanie na obiekt regulacji. Równolegle STM32 wysyła do aplikacji ramki telemetryczne zawierające pomiar, uchyb, wartość zadaną i sygnał sterujący, co umożliwia wizualizację oraz strojenie regulatora po stronie PC.

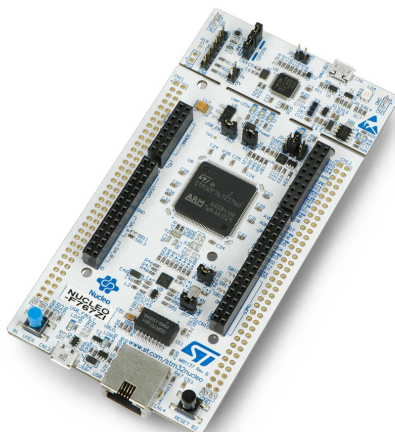


Rysunek 2.2: Schemat blokowy stanowiska: połączenia podsystemów i przepływ sygnałów (elektrycznych, optycznych i mechanicznych) między aplikacją PC, mikrokontrolerem STM32, sensorami, serwomechanizmem oraz obiektem regulacji

2.2 Mikrokontroler STM32 i peryferia

Mikrokontroler STM32F767ZI

Głównym procesorem stanowiska jest mikrokontroler STM32F767ZI [2] –rys. 2.3 zamontowany na płytce ewaluacyjnej Nucleo-F767ZI. Układ oparty jest na rdzeniu ARM Cortex-M7 taktowanym z częstotliwością 216 MHz i wyposażony jest w bogatą gamę peryferiów: kilka interfejsów I²C i UART, timery sprzętowe, przetwornik ADC oraz obsługę sygnałów PWM.



Rysunek 2.3: Płytką ewaluacyjna STM32 Nucleo-F767ZI

Serwomechanizm MG946R

Do sterowania kątem belki zastosowano serwomechanizm TowerPro MG946R –rys. 2.4 które, przyjmuje sygnał PWM o częstotliwości 50 Hz i czasie wypełnienia od 1 ms (pełne wychylenie w jedną stronę) do 2 ms (pełne wychylenie w drugą stronę). Zakres roboczy ograniczony jest w oprogramowaniu do 60–140° w celu zabezpieczenia mechaniki.



Rysunek 2.4: Serwomechanek TowerPro MG946R

Kamera systemu wizyjnego

Do rejestracji obrazu zastosowano kamerę USB o rozdzielczości 640×480 pikseli pracującą z częstotliwością 30 klatek na sekundę. Kamera umieszczona jest prostopadle do belki, obejmując całą jej długość wraz z narożnymi znacznikami ArUco. Obraz pozyskiwany jest przez bibliotekę OpenCV za pomocą interfejsu `cv2.VideoCapture`, a aplikacja umożliwia wybór indeksu kamery z listy rozwijanej w panelu wizyjnym.



Rysunek 2.5: Kamera USB używana w systemie wizyjnym

Potencjometr

Potencjometr liniowy podłączony do wejścia analogowego PA0 (ADC1_CH0) umożliwia ręczne zadawanie wartości referencyjnej pozycji kulki w trybie analogowym. Przetwornik ADC skonfigurowany jest z rozdzielczością 12-bitową (0–4095), co odpowiada zakresowi pozycji 0–250 mm.

2.3 Środowisko programistyczne

2.3.1 Firmware STM32

Oprogramowanie mikrokontrolera zostało opracowane w środowisku STM32CubeIDE (Eclipse CDT ze zintegrowanym kompilatorem ARM GCC). Konfigurację peryferiów (timery, I²C, UART, ADC, PWM) wygenerowano za pomocą narzędzia STM32CubeMX. Kod źródłowy podzielono na osobne moduły C zgodnie ze standardem Doxygen:

Tabela 2.2: Moduły firmware

Moduł	Opis
main.c/h	Główna aplikacja, konfiguracja peryferiów
servo_pid.c/h	Regulator PID z anti-windup
lqr.c/h	Regulator LQR (przestrzeń stanów)
filters.c/h	Filtry cyfrowe (EMA, mediana, 1-Euro)
crc8.c/h	Suma kontrolna CRC-8
calibration.c/h	Kalibracja czujnika VL53L0X
vl53l0x.c/h	Sterownik czujnika (I ² C)

System operacyjny czasu rzeczywistego FreeRTOS zarządza wykonaniem dwóch wątków (tasków):

- `ControlTask` (priorytet wysoki) — pętla sterowania;
- `DefaultTask` (priorytet normalny) — obsługa komend UART.

2.3.2 Aplikacja desktopowa

Aplikacja nadzorcza napisana jest w języku Python z wykorzystaniem frameworka PySide6 (Qt 6). Zapewnia ona:

- połączenie z mikrokontrolerem przez port szeregowy (pyserial),
- wizualizację sygnałów w czasie rzeczywistym (pyqtgraph),
- zadawanie wartości referencyjnej i nastaw regulatorów,
- nagrywanie danych i eksport do pliku CSV,
- podgląd kamery z detekcją pozycji kulki (OpenCV).

2.4.2 Ograniczenie zakresu serwa

Wyjście regulatora jest nasycone do bezpiecznego zakresu kąta serwa:

```
1 #define SERVO_MIN_LIMIT 60.0f
2 #define SERVO_MAX_LIMIT 140.0f
3
4 if (pid_angle < SERVO_MIN_LIMIT) pid_angle = SERVO_MIN_LIMIT;
5 if (pid_angle > SERVO_MAX_LIMIT) pid_angle = SERVO_MAX_LIMIT;
6 SetServoAngle(pid_angle);
```

Listing 2.2: Saturacja wyjścia i ograniczenia serwa (main.c)

2.4.3 Protokół telemetry UART z sumą kontrolną CRC-8

Dane pomiarowe przesyłane są przez UART 3 (115200 baud) w formacie tekstowym z sumą kontrolną CRC-8 (wielomian 0x07). Format ramki telemetry wysyłanej przez mikrokontroler:

T:%lu;D:%d;Z:%d;A:%d;F:%d;E:%d;V:%d;B:%d;C:%02X\r\n

gdzie pola oznaczają odpowiednio: znacznik czasu (ms), pozycja kulki z wizji, setpoint, kąt serwa, pozycja filtrowana, uchyb, średni uchyb, kąt belki z wizji ($\times 100$) i CRC-8.

```
1 uint8_t CalculateCRC8(const char *data, int len) {
2     uint8_t crc = 0x00;
3     for (int i = 0; i < len; i++) {
4         crc ^= data[i];
5         for (uint8_t j = 0; j < 8; j++) {
6             if (crc & 0x80)
7                 crc = (crc << 1) ^ 0x07;
8             else
9                 crc <<= 1;
10        }
11    }
12    return crc;
13 }
```

Listing 2.3: Implementacja CRC-8 (crc8.c)

2.4.4 Zadania FreeRTOS

System operacyjny czasu rzeczywistego tworzy dwa wątki:

```
1 osThreadDef(defaultTask, StartDefaultTask,
2             osPriorityNormal, 0, 256);
3 defaultTaskHandle = osThreadCreate(osThread(defaultTask), NULL);
4
5 osThreadDef(ControlTask, StartControlTask,
6             osPriorityHigh, 0, 1024);
7 ControlTaskHandle = osThreadCreate(osThread(ControlTask), NULL);
8
```

```
9 osKernelStart();
```

Listing 2.4: Tworzenie zadań FreeRTOS (main.c)

2.5 System wizyjny

Stanowisko wyposażone jest w podsystem wizji komputerowej. Zastosowanie kamery pozwala na obserwację całej długości obiektu i jednocześnie wyznaczanie geometrii belki na podstawie znaczników odniesienia. Przetwarzanie obrazu realizowane jest po stronie aplikacji desktopowej w języku Python z użyciem biblioteki OpenCV, natomiast mikrokontroler STM32 otrzymuje już gotowe wielkości fizyczne: pozycję kulki w milimetrach oraz kąt belki w stopniach.

Algorytm wizyjny został umieszczony w klasie `OpenCVPanel`. Panel aplikacji zawiera trzy tryby podglądu: detekcję kulki, detekcję znaczników ArUco oraz widok pomiarowy łączący oba algorytmy. Dzięki temu możliwe jest osobne strojenie progów kolorystycznych, parametrów markerów oraz kalibracji geometrycznej, a następnie obserwacja końcowego wyniku wykorzystywanego przez regulator.

2.5.1 Konfiguracja kamery i akwizycja obrazu

Akwizycja obrazu realizowana jest za pomocą funkcji `cv2.VideoCapture` z biblioteki OpenCV. Kamera konfigurowana jest na stałą rozdzielczość 640×480 pikseli i częstotliwość 30 FPS z minimalnym buforem klatek (`CAP_PROP_BUFFERSIZE = 1`) w celu minimalizacji opóźnienia pomiarowego. Aplikacja obsługuje automatyczne wykrywanie dostępnych kamer (tryb auto) lub ręczny wybór indeksu urządzenia przez użytkownika. Przetwarzanie kolejnych klatek wyzwala jest przez timer Qt uruchamiany z okresem 1 ms (`camera_timer.start(1)`), dzięki czemu aplikacja umożliwia odebranie i przetworzenie obrazu tak szybko, jak pozwala na to rzeczywista liczba klatek dostarczana przez kamerę i czas obliczeń algorytmu.

Po uruchomieniu kamery aplikacja wykonuje próbny odczyt klatki. Dopiero poprawnie odebrany obraz powoduje przejście panelu w stan aktywny. Każda przetworzona klatka jest konwertowana z formatu OpenCV do obiektu `QImage`, a następnie wyświetlana w interfejsie Qt razem z bieżącą informacją o liczbie klatek na sekundę.

2.5.2 Detekcja kulki metodą progowania w przestrzeni HSV

Kulka (piłeczka ping-pongowa) wykrywana jest na podstawie koloru w przestrzeni barw HSV (*Hue, Saturation, Value*). Podejście to wybrano ze względu na niewielki koszt obliczeniowy oraz fakt, że obiekt ma wyróżniający się kolor i regularny kształt. Przestrzeń HSV jest wygodniejsza od RGB, ponieważ składowa barwy jest w dużym stopniu oddzielona od jasności, co ułatwia dobór progów przy umiarkowanych zmianach oświetlenia. Przetwarzanie każdej klatki przebiega w następujących krokach:

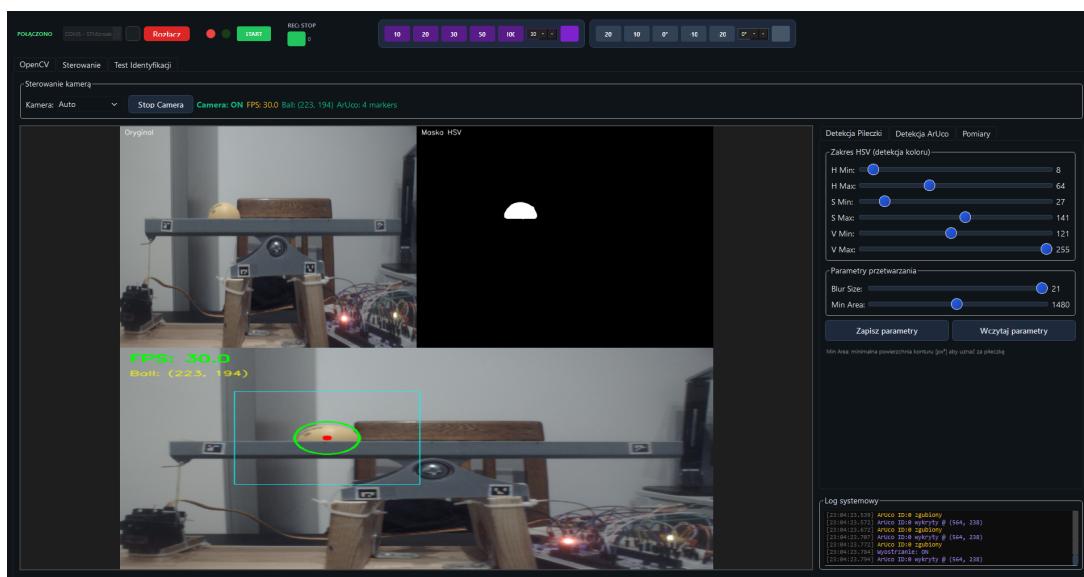
1. rozmycie gaussowskie redukujące lokalny szum obrazu; w zapisanej konfiguracji używane jest jądro 21×21 px,

2. konwersja obrazu z formatu BGR do HSV za pomocą `cv2.cvtColor`,
3. progowanie zakresu barwy funkcją `cv2.inRange`: $H \in [8, 64]$, $S \in [27, 141]$, $V \in [121, 255]$.
4. operacje morfologiczne na jądrze eliptycznym 5×5 px: domknięcie (`MORPH_CLOSE`) wypełnia niewielkie przerwy w masce, a otwarcie (`MORPH_OPEN`) usuwa pojedyncze artefakty tła,
5. wykrywanie konturów funkcją `cv2.findContours`; wybierany jest kontur o największym polu, a po przekroczeniu progu minimalnego 1480 px^2 wyznaczane jest minimalne koło okalające (`cv2.minEnclosingCircle`).

Wartość progu odpowiada okręgowi o promieniu ok. 22 px, czyli ok. 75% pola spodziewanego konturu kulki o średnicy 40 mm w obrazie kamery. Stanowi to próg odrzucający mniejsze artefakty barwne pochodzące od elementów konstrukcyjnych i refleksów, przy zachowaniu marginesu na zniekształcenia maski na krawędziach kulki.

Środek wyznaczonego koła traktowany jest jako pozycja kulki w układzie pikselowym kamery. Aby ograniczyć czas przetwarzania, zastosowano obszary zainteresowania (ROI). Po poprawnej detekcji kolejna klatka analizowana jest tylko w oknie o marginesie 100 px wokół ostatniej pozycji kulki. Jeżeli kontur nie zostanie znaleziony albo jego pole jest zbyt małe, dynamiczny ROI jest zerowany i algorytm wraca do przeszukiwania całego obrazu.

Parametry progowania HSV, rozmiar filtra rozmywającego oraz próg minimalnego pola konturu mogą być aktualizowane z poziomu GUI. Ustawienia są zapisywane do pliku `opencv_params.json` i wczytywane automatycznie przy starcie aplikacji, co pozwala szybko odtworzyć dobraną konfigurację po ponownym uruchomieniu programu. Podgląd maski HSV oraz obrazu wynikowego z zaznaczoną kulką przedstawiony jest na rys. –rys. 2.7.



Rysunek 2.7: Widok zakładki detekcji kulki

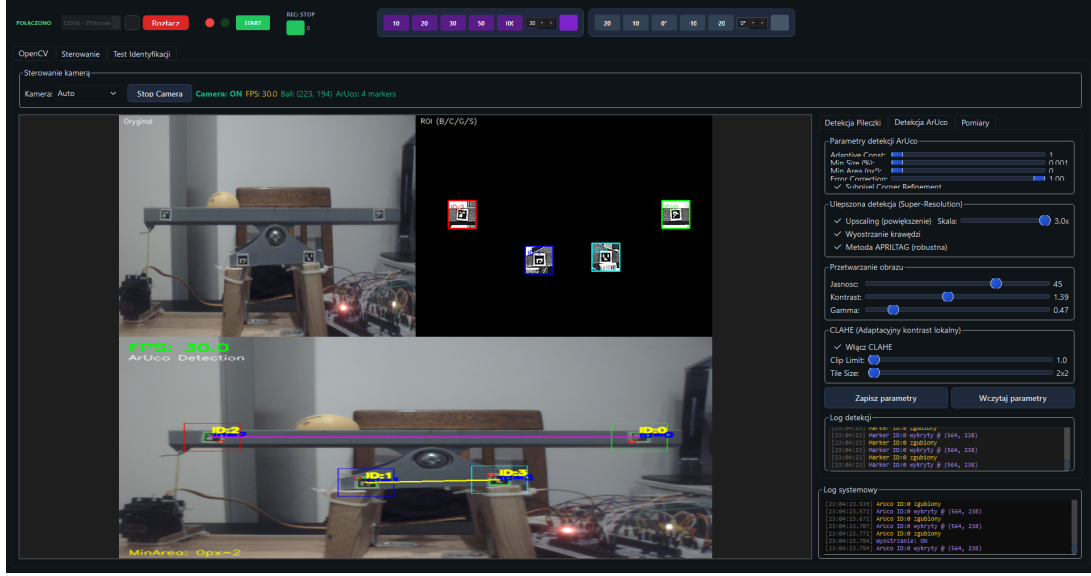
2.5.3 Wyznaczanie pozycji belki na podstawie znaczników ArUco

Na ramie stanowiska przyklejone są cztery kwadratowe znaczniki ze słownika DICT_4X4_50: ID 0 (prawy koniec belki), ID 1 i ID 3 (punkty referencyjne ramy) oraz ID 2 (lewy koniec belki). Detekcja znaczników realizowana jest przez moduł `cv2.aruco.ArucoDetector` z parametrami dobranymi pod niewielkie znaczniki widoczne w obrazie kamery. Dla każdego wykrytego markera obliczany jest środek jako średnia współrzędnych przeciwległych narożników. Odcinek łączący środki ID 2 oraz ID 0 definiuje oś belki, natomiast odcinek ID 1–ID 3 może pełnić funkcję nieruchomej linii referencyjnej.

W celu zwiększenia odporności detekcji na cienie i mały rozmiar markerów zastosowano następujące mechanizmy:

- adaptacyjne wyrównanie histogramu CLAHE stosowane do wycinków obrazu, dzięki czemu lokalnie zwiększany jest kontrast bez nadmiernego wzmacniania kontrastu w jasnych obszarów,
- regulację jasności, kontrastu i korekcji gamma z poziomu panelu aplikacji,
- opcjonalne powiększenie analizowanego wycinka przed detekcją oraz wyostanie krawędzi filtrem konwolucyjnym 3×3 ,
- doprecyzowanie położenia narożników metodą `CORNER_REFINE_APRILTAG` lub, w starszych wersjach OpenCV, metodą subpikselową,
- złagodzone ograniczenia geometryczne detektora, w tym mniejszy minimalny obwód markera i większą dopuszczalną deformację czworokąta.

Każdy ze znaczników śledzony jest w osobnym dynamicznym ROI. Po pierwszej poprawnej detekcji algorytm w kolejnych klatkach przeszukuje małe okno o marginesie 30 px wokół ostatniej pozycji danego markera. Jeśli marker nie zostanie odnaleziony przez kilka klatek, aplikacja wraca do szerszego statycznego ROI. Dodatkowo w trybie pomiarowym działa mechanizm podtrzymania pozycji: chwilowo zgubiony marker może zachować ostatnie poprawne współrzędne maksymalnie przez 15 klatek. Ogranicza to pojedyncze skoki kąta belki wynikające z krótkotrwałego zasłonięcia lub błędu detekcji. Rozmieszczenie znaczników na stanowisku przedstawione jest na rys. –rys. 2.8.



Rysunek 2.8: Stanowisko z nałożonymi wykrytymi znacznikami ArUco: ID 2 (lewy koniec), ID 0 (prawy koniec), ID 1 i ID 3 (punkty referencyjne ramy)

2.5.4 Kalibracja geometryczna i przeliczanie piksel $\rightarrow$ mm

Pozycja kulki wyznaczana jest jako rzut prostokątny jej środka na oś belki (odcinek łączący środki znaczników ID 2 i ID 0):

$$t = \frac{(\mathbf{p} - \mathbf{A}) \cdot (\mathbf{B} - \mathbf{A})}{\|\mathbf{B} - \mathbf{A}\|^2}, \quad t \in [0, 1], \quad (2.1)$$

gdzie $\mathbf{p}$ to wektor współrzędnych środka kulki w pikselach, $\mathbf{A}$ i $\mathbf{B}$ to odpowiednio wektory współrzędnych środków ID 2 i ID 0. W implementacji parametr t jest ograniczany do przedziału $[0, 1]$, dlatego wynik odpowiada położeniu na odcinku belki nawet wtedy, gdy środek kulki zostanie wyznaczony nieznacznie poza linią markerów.

Parametr t przeliczany jest na milimetry za pomocą kalibracji dwupunktowej: użytkownik umieszcza kulkę kolejno przy lewym i prawym końcu belki, rejestrując wartości $t_{\min} = 0,039$ i $t_{\max} = 0,990$. Wartości te pochodzą z aktualnego pliku konfiguracyjnego aplikacji i mogą zostać ponownie wyznaczone po zmianie położenia kamery lub markerów. Wzór konwersji ma postać:

$$x_{\text{mm}} = \frac{t - t_{\min}}{t_{\max} - t_{\min}} \cdot L, \quad (2.2)$$

gdzie $L = 250$ mm jest długością aktywnej części belki.

Kąt nachylenia belki wyznaczany jest jako kąt odcinka ID 2–ID 0 względem poziomu obrazu (metoda domyślna) lub względem linii referencyjnej ID 1–ID 3 (metoda alternatywna dostępna z GUI). W pierwszej metodzie wykorzystywana jest funkcja `atan2`, przy czym znak składowej pionowej jest odwracany ze względu na układ współrzędnych obrazu, który jest lewoskrętny. W drugiej metodzie obliczany jest kąt między dwoma wektorami: osią belki oraz linią referencyjną. GUI umożliwia także zapisanie offsetu kąta, dzięki czemu

aktualne ustawienie belki może zostać przyjęte jako 0° . Parametry kalibracji zapisywane są do pliku `opencv_params.json` i wczytywane automatycznie przy kolejnym uruchomieniu aplikacji.

2.5.5 Protokół komunikacyjny PC → STM32

Wyniki pomiaru (pozycja kulki i kąt belki) przesyłane są do mikrokontrolera przez ten sam port szeregowy UART (115200 baud), którym odbywa się telemetria z STM32. Format ramki wizyjnej:

`V:%d.%d;B:%d.%d;C:%02X\n`

gdzie *V* to pozycja kulki w mm, *B* to kąt belki w stopniach, a *C* to suma kontrolna CRC-8 (wielomian 0x07) obejmująca pola *V* i *B*. Ramki wysyłane są z częstotliwością około 30 Hz przez dedykowany timer Qt, ale tylko wtedy, gdy pozycja kulki została poprawnie wyznaczona. Wysyłanie realizuje metoda `send_vision_data()` z modułu `serial_manager.py`, która oblicza CRC tym samym algorytmem co firmware STM32.

Po stronie firmware funkcja `ParseVisionFrame()` (plik `main.c`) odróżnia ramkę wizyjną od standardowych komend sterujących po pierwszym znaku *V*, waliduje CRC i aktualizuje zmienne globalne `g_vision_ball_pos` i `g_vision_beam_angle`. Jeśli przez ponad 200 ms nie nadejdzie żadna poprawna ramka wizyjna, regulator przechodzi w tryb awaryjny: jako pomiar pozycji używana jest ostatnia poprawna wartość, a kąt belki przyjmuje wartość 0° .

W pętli sterowania STM32 pozycja z systemu wizyjnego jest dodatkowo filtrowana prostym filtrem wykładniczym. Tak przygotowana wartość zastępuje pomiar z czujnika odległości i jest używana do obliczenia uchybu regulatora. W trybie LQR do wektora stanu trafia również kąt belki przesłany w ramce wizyjnej.

```

1 void ParseVisionFrame(const char *frame) {
2     // ...parsowanie pol V i B...
3     if (crc_found && ball_pos >= 0.0f) {
4         uint8_t crc_calc = CalculateCRC8(frame, data_len);
5         if (crc_calc == crc_received) {
6             g_vision_ball_pos = ball_pos;
7             g_vision_beam_angle = beam_angle;
8             g_vision_data_valid = 1;
9             g_vision_last_update = HAL_GetTick();
10        }
11    }
12 }

```

Listing 2.5: Fragment funkcji `ParseVisionFrame()` (`main.c`) – parsowanie i walidacja CRC

Model matematyczny i projekt regulacji

Rozdział ten przedstawia wyprowadzenie modelu matematycznego układu kulka na belce, metodę identyfikacji jego parametrów oraz projekt regulatorów PID i LQR.

3.1 Model matematyczny obiektu regulacji

Układ składa się z trzech elementów połączonych szeregowo: serwomechanizmu, mechanicznego połączenia dźwigniowego oraz kulki toczącej się po belce.

3.1.1 Dynamika serwomechanizmu

Dynamikę serwomechanizmu w pierwszym przybliżeniu opisuje człon inercyjny pierwszego rzędu:

$$G_{\text{serwo}}(s) = \frac{\Theta_{\text{serwo}}(s)}{\Theta_{\text{ref}}(S)} = \frac{1}{T_{\text{serwo}} s + 1}, \quad (3.1)$$

gdzie T_{serwo} jest stałą czasową serwa.

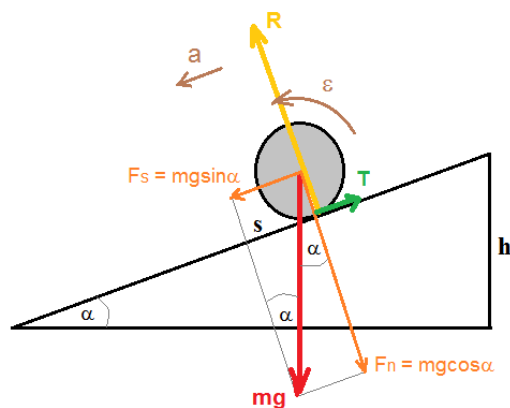
Na podstawie danych katalogowych serwomechanizmu MG946R (0,20 s/60° przy 4,8 V; 0,11 s/60° przy 6 V) interpolując dla napięcia 5 V:

$$V_{5V} \approx 0,20 - \frac{0,09}{1,2} \cdot 0,2 \approx 0,185 \text{ s/60}^\circ. \quad (3.2)$$

Przyjęto stałą czasową $T_{\text{serwo}} = 0,095 \text{ s}$.

3.1.2 Dynamika kulki na belce

Równanie ruchu — II zasada dynamiki



Rysunek 3.1: Rozkład sił działających na kulkę na równi pochyłej

Dla kulki toczącej się bez poślizgu po belce pochylonej pod kątem θ równanie ruchu postępowego wynosi:

$$m\ddot{x} = m g \sin \theta - T, \quad (3.3)$$

gdzie:

- x - pozycja
- g - przyspieszenie ziemskie
- θ - kąt nachylenia równi

a równanie momentu obrotowego wokół osi kulki:

$$T \cdot R = J \ddot{\alpha}, \quad (3.4)$$

gdzie:

- T — siła tarcia statycznego
- R — promień kulki
- J — moment bezwładności
- $\epsilon = \ddot{\alpha}$ — przyspieszenie kątowe kulki

Warunek toczenia bez poślizgu

Z warunku $\ddot{x} = \ddot{\alpha} \cdot R$ eliminuje się siłę tarcia:

$$m\ddot{x} + \frac{J}{R^2}\ddot{x} = m g \sin \theta \quad \Rightarrow \quad \ddot{x} = \frac{g \sin \theta}{1 + J/(mR^2)}. \quad (3.5)$$

Dla małych kątów $\sin \theta \approx \theta$, a dla kulki $J = \frac{2}{3}mR^2$, stąd:

$$\ddot{x} = \underbrace{\frac{g}{1 + 2/3}}_{c_{\text{ball}}} \cdot \theta = 0,6 g \cdot \theta. \quad (3.6)$$

Stała dynamiki kulki wynosi:

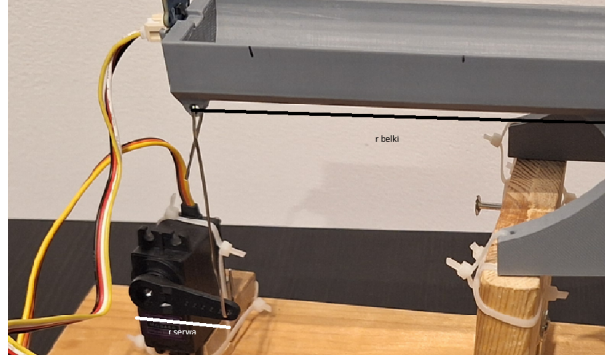
$$c_{\text{ball}} = 0,6 \times 9,81 = 5,886 \text{ m/s}^2. \quad (3.7)$$

Transmitancja kulki na belce

Po zastosowaniu względem 3.6 transformaty Laplace'a (zerowe warunki początkowe) otrzymuje się:

$$G_{\text{ball}}(s) = \frac{X(s)}{\Theta_{\text{beam}}(s)} = \frac{c_{\text{ball}}}{s^2}. \quad (3.8)$$

3.1.3 Połączenie mechaniczne i transmitancja całkowita



Rysunek 3.2: Schemat mechanicznego połączenia serwa z belką (dźwignia)

Kąty serwa i belki powiązane są przez przekładnię mechaniczną:

$$\theta_{\text{beam}} = \frac{r_{\text{serwo}}}{r_{\text{belka}}} \cdot \theta_{\text{serwo}} = k_{\text{mech}} \cdot \theta_{\text{serwo}}, \quad (3.9)$$

gdzie $r_{\text{serwo}} = 3 \text{ cm}$, $r_{\text{belka}} = 15 \text{ cm}$, zatem $k_{\text{mech}} = 0,2$.

Całkowita transmitancja układu (od sygnału sterującego $U(s)$ do pozycji kulki $X(s)$):

$$G_{\text{total}}(s) = k_{\text{mech}} \cdot G_{\text{serwo}}(s) \cdot G_{\text{ball}}(s) = \frac{k_{\text{mech}} \cdot c_{\text{ball}}}{s^2(T_{\text{serwo}} s + 1)}. \quad (3.10)$$

Podstawiając wartości liczbowe:

$$G_{\text{total}}(s) = \frac{0,2 \cdot 5,886}{0,095 s^3 + s^2} = \frac{1,177}{0,095 s^3 + s^2}. \quad (3.11)$$

3.2 Identyfikacja parametrów modelu

3.2.1 Identyfikacja wzmocnienia metodą odpowiedzi skokowej

Metodyka pomiarowa

W celu doświadczalnego wyznaczenia wzmocnienia statycznego układu przeprowadzono eksperyment polegający na rejestracji odpowiedzi pozycji kulki na wymuszenia skokowe kąta serwomechanizmu. Punktem pracy (poziomem zerowym) przyjęto wysterowanie serwomechanizmu równe 100° , przy którym belka pozostaje w przybliżeniu pozioma. Na wejście układu podawano kolejno skoki do pozycji $109^\circ, 112^\circ, 115^\circ, 120^\circ, 125^\circ, 130^\circ$ oraz 140° . Amplituda A każdego wymuszenia stanowiła różnicę między zadaną pozycją a poziomem zerowym (np. dla 125° wynosiła $A = 25^\circ$). W trakcie eksperymentu rejestrowano pozycję kulki na belce wyrażoną w milimetrach w funkcji czasu.

Analiza danych i estymacja wzmocnienia

Dane pomiarowe zapisane w plikach CSV poddano obróbce w środowisku MATLAB. W pierwszym kroku zwizualizowano surowe przebiegi i ręcznie wyznaczono okno czasowe odpowiadające właściwemu ruchowi kulki – od momentu inicjacji skoku do chwili uderzenia kulki w ogranicznik belki.

Zgodnie z uproszczonym modelem (zob. równanie (??)) przyjęto, że transmitancja łącząca kąt pochylenia belki z pozycją kulki ma postać podwójnego całkowania:

$$G(s) = \frac{K}{s^2}. \quad (3.12)$$

W dziedzinie czasu odpowiada to równaniu różniczkowemu $\ddot{y}(t) = K \cdot u(t)$. Przyłożenie na wejście funkcji skokowej o amplitudzie A [°] i dwukrotne scałkowanie (przy zerowych warunkach początkowych) daje:

$$y(t) = \frac{1}{2} K A t^2, \quad y \text{ [mm]}, \quad t \text{ [s]}. \quad (3.13)$$

Wycięty fragment danych scentrowano w punkcie (0, 0), a następnie za pomocą funkcji `polyfit` wykonano aproksymację wielomianem drugiego stopnia $y(t) = p_1 t^2 + p_2 t + p_3$. Porównując współczynnik przy t^2 z równaniem (3.13) otrzymuje się:

$$p_1 = \frac{1}{2} K A \quad \implies \quad K = \frac{2p_1}{A}, \quad K \left[\frac{\text{mm}}{\text{s}^2 \cdot ^\circ} \right]. \quad (3.14)$$

Poniższy listing przedstawia fragment skryptu realizującego opisaną procedurę:

Listing 3.1: Fragment skryptu MATLAB estymującego wzmocnienie K

```
% Ograniczenie danych do właściwego ruchu pilki
idx = t_raw > 1.1 & t_raw < 2.2;
t = t_raw(idx);
y = y_raw(idx);
```

```
% Przesunięcie czasu i pozycji do zera
t = t - t(1);
y = y - y(1);

% Dopasowanie wielomianu 2. stopnia
p = polyfit(t, y, 2);

% Obliczenie wzmocnienia K [mm / (s^2 * deg)]
K_obl = (2 * p(1)) / A_stopnie;
```

Weryfikacja wyników i wyznaczenie modelu końcowego

Powyższą procedurę powtórzono dla wszystkich siedmiu wartości wychylenia. Rysunki – rys. 3.3—rys. 3.7 przedstawiają surowe przebiegi pozycji kulki oraz dopasowane parabole dla kolejnych prób.

Zaobserwowano, że dla dużych kątów wychylenia estymowane wzmocnienie w największym stopniu odbiega od wartości średniej. Przyczyną jest niezerowa dynamika serwomechanizmu – przy dużych skokach zadanej pozycji kulka zaczyna się toczyć zanim serwo osiągnie wartość docelową, co narusza założenie o skokowym charakterze wymuszenia i zaburza wyniki estymacji. Z tego powodu z analizy wykluczono próby dla wychyleń 130° i 140°, pozostawiając pięć zestawów danych.

Wyznaczone wartości wzmocnienia dla pięciu uwzględnionych prób wynoszą:

$$K_i = [11,9254; 11,2886; 12,2396; 12,2772; 11,9280] \left[\frac{\text{mm}}{\text{s}^2 \cdot ^\circ} \right]. \quad (3.15)$$

Kończącą wartość wzmocnienia przyjęto jako średnią arytmetyczną powyższych próbek:

$$\bar{K} = \frac{1}{n} \sum_{i=1}^n K_i = 11,9318 \frac{\text{mm}}{\text{s}^2 \cdot ^\circ}. \quad (3.16)$$

Aby ocenić rozrzut otrzymanych wartości, wyznaczono także wariancję próbkową oraz odchylenie standardowe:

$$s^2 = \frac{1}{n-1} \sum_{i=1}^n (K_i - \bar{K})^2 = 0,1570 \left[\frac{\text{mm}}{\text{s}^2 \cdot ^\circ} \right]^2, \quad (3.17)$$

$$s = \sqrt{s^2} = 0,3962 \frac{\text{mm}}{\text{s}^2 \cdot ^\circ}. \quad (3.18)$$

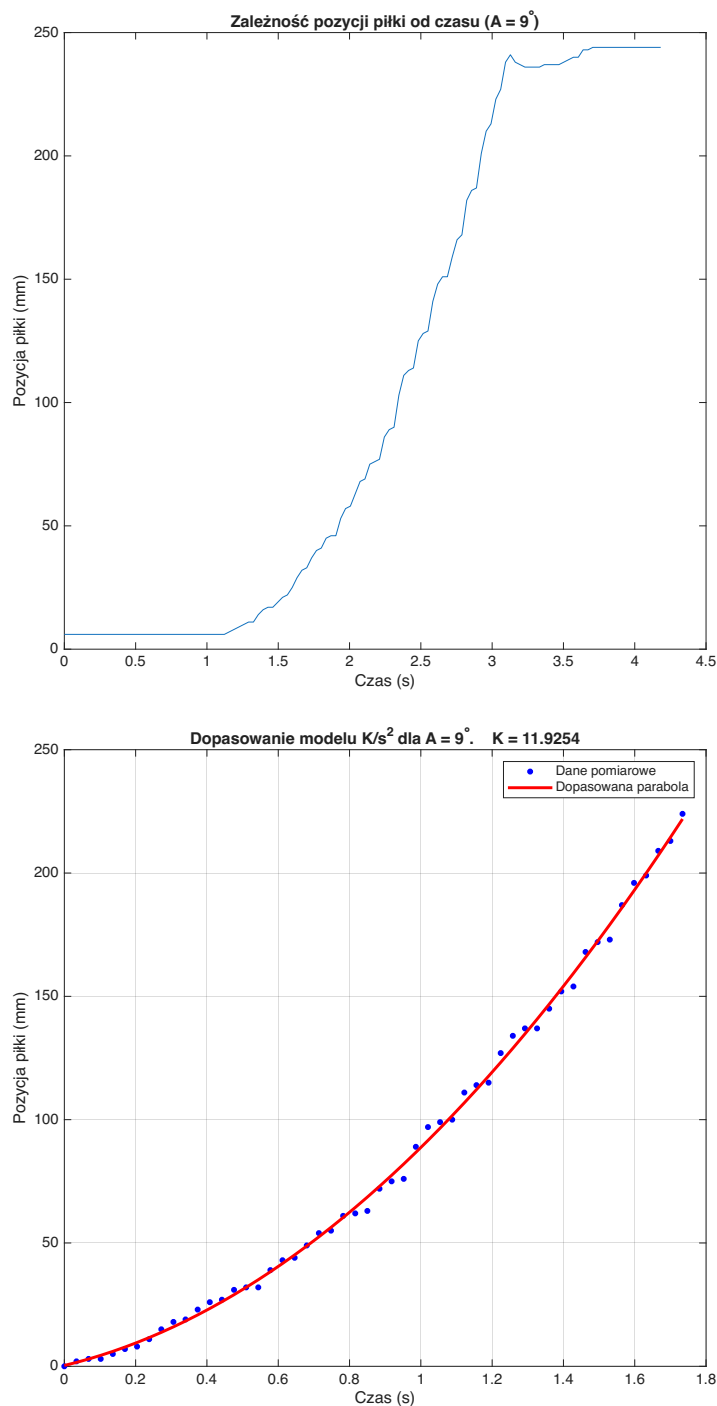
Odchylenie standardowe stanowi ok. 3,3% wartości średniej, co świadczy o dobrej powtarzalności estymacji wzmocnienia w obrębie uwzględnionych prób.

Transmitancja identyfikowanego układu kulka–belka przyjmuje zatem postać:

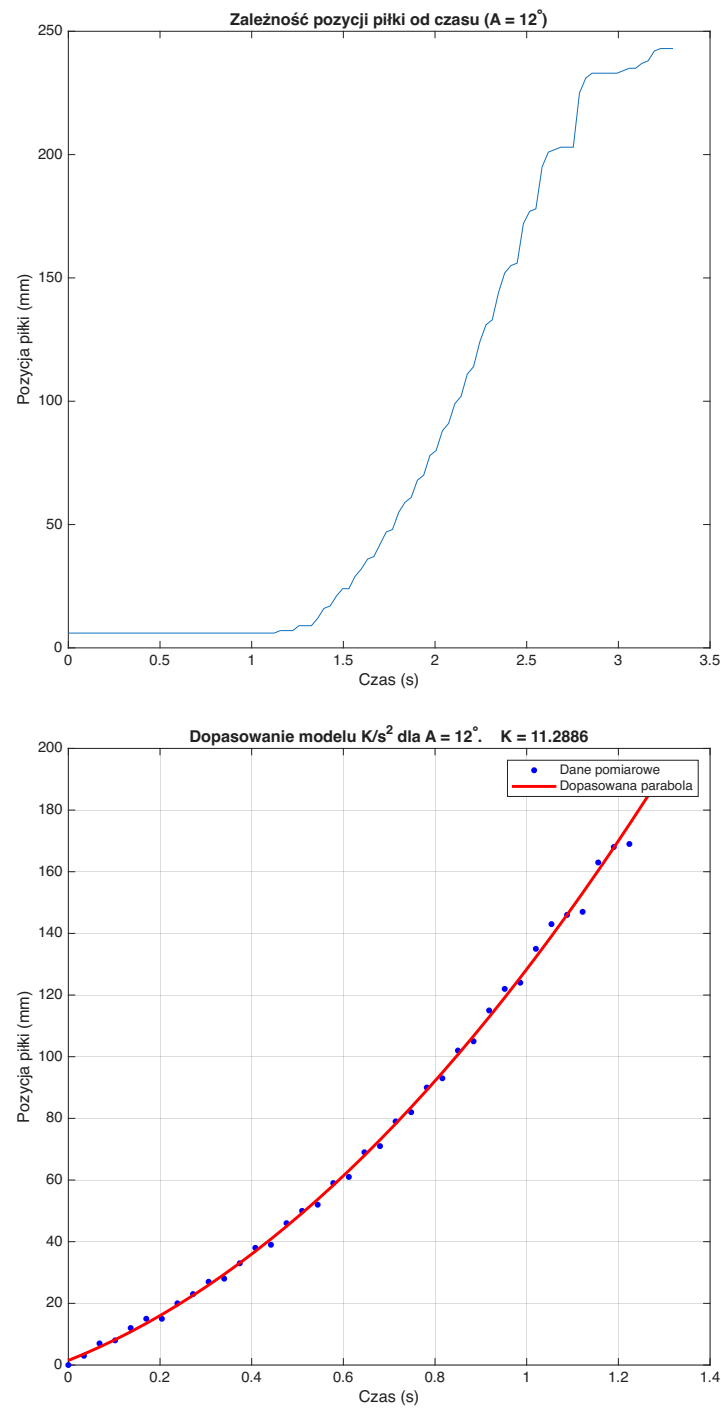
$$G(s) = \frac{11,9318}{s^2}, \quad \left[\frac{\text{mm}}{\text{s}^2 \cdot ^\circ} \right]. \quad (3.19)$$

Należy odnotować, że na dokładność estymacji wpływ ma ograniczona liczba prób pomiarowych oraz ręczny dobór okna czasowego analizy – dobór granic okna istotnie

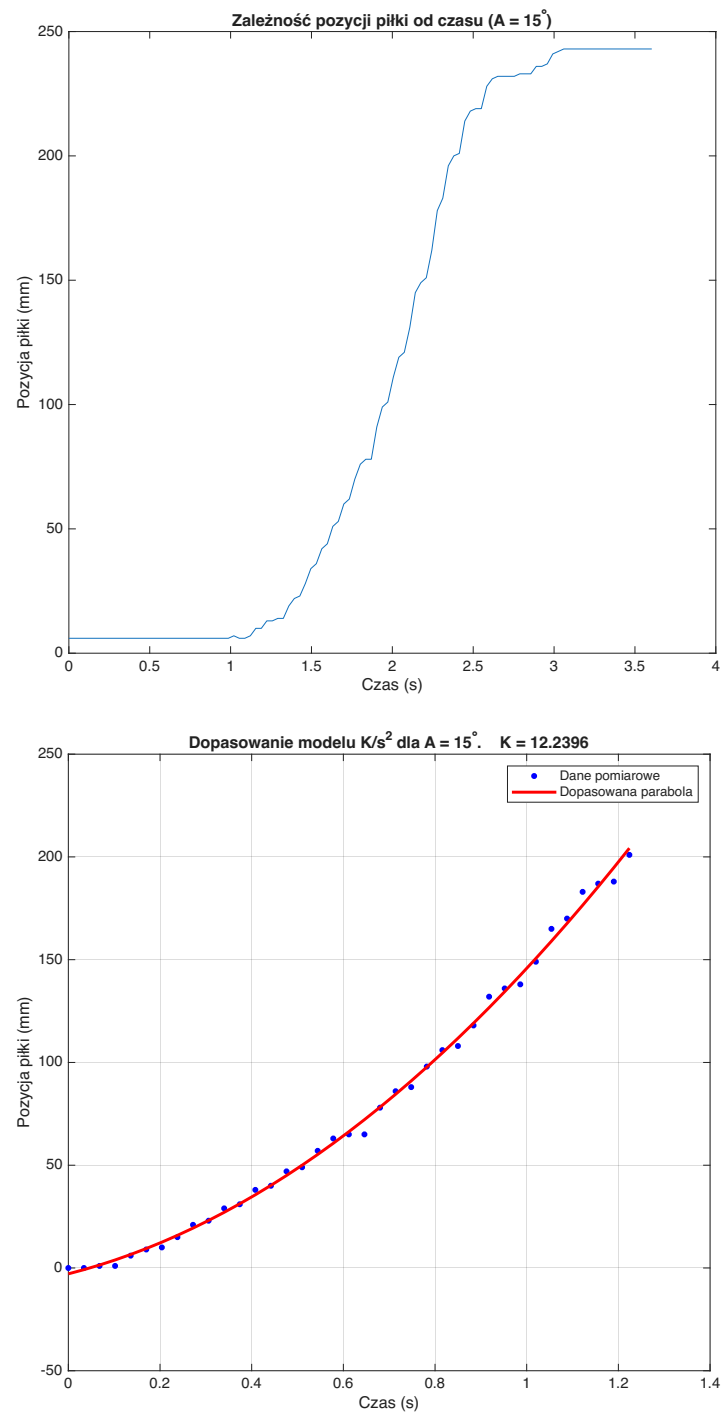
oddziałuje na wyznaczoną wartość współczynnika p_1 , a tym samym na końcowy wynik uśredniania.



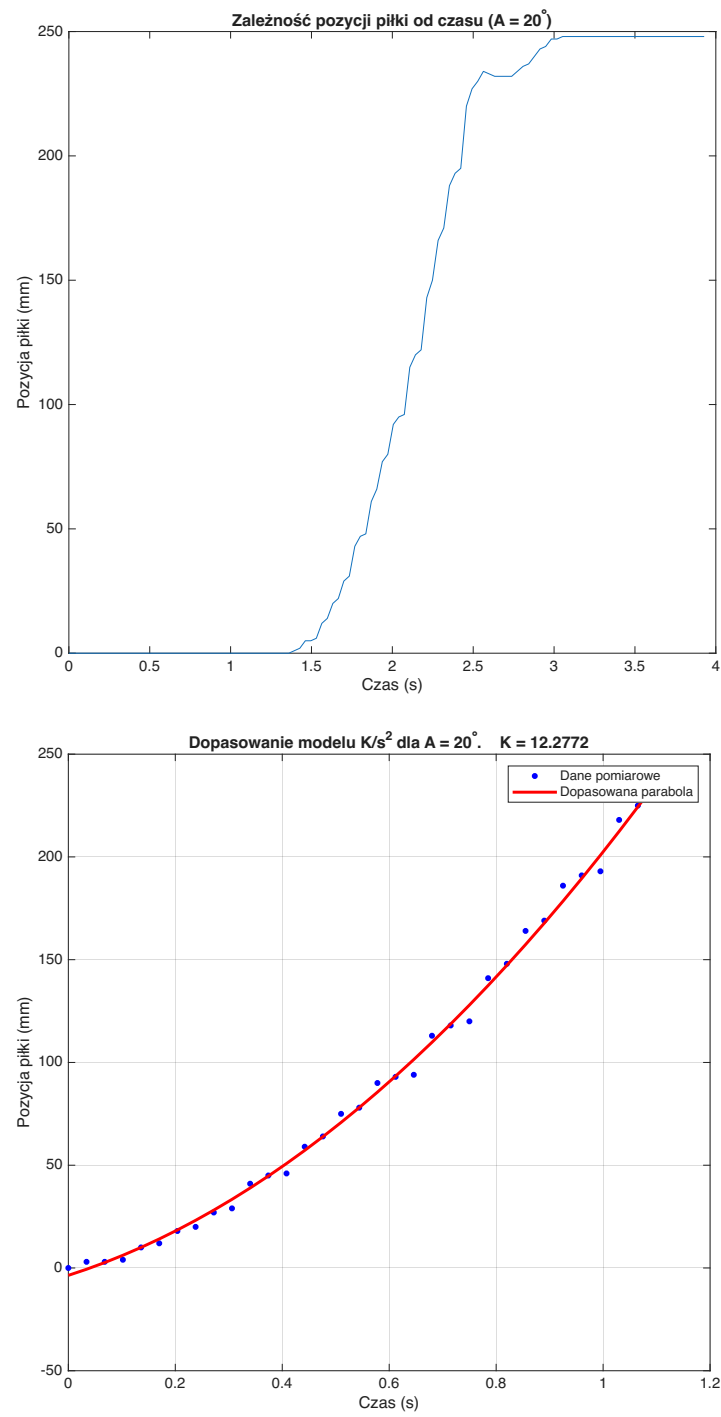
Rysunek 3.3: Wyniki identyfikacji dla pozycji zadanej 109° : surowe dane pozycji kulki (góra) oraz dopasowanie paraboliczne (dół).



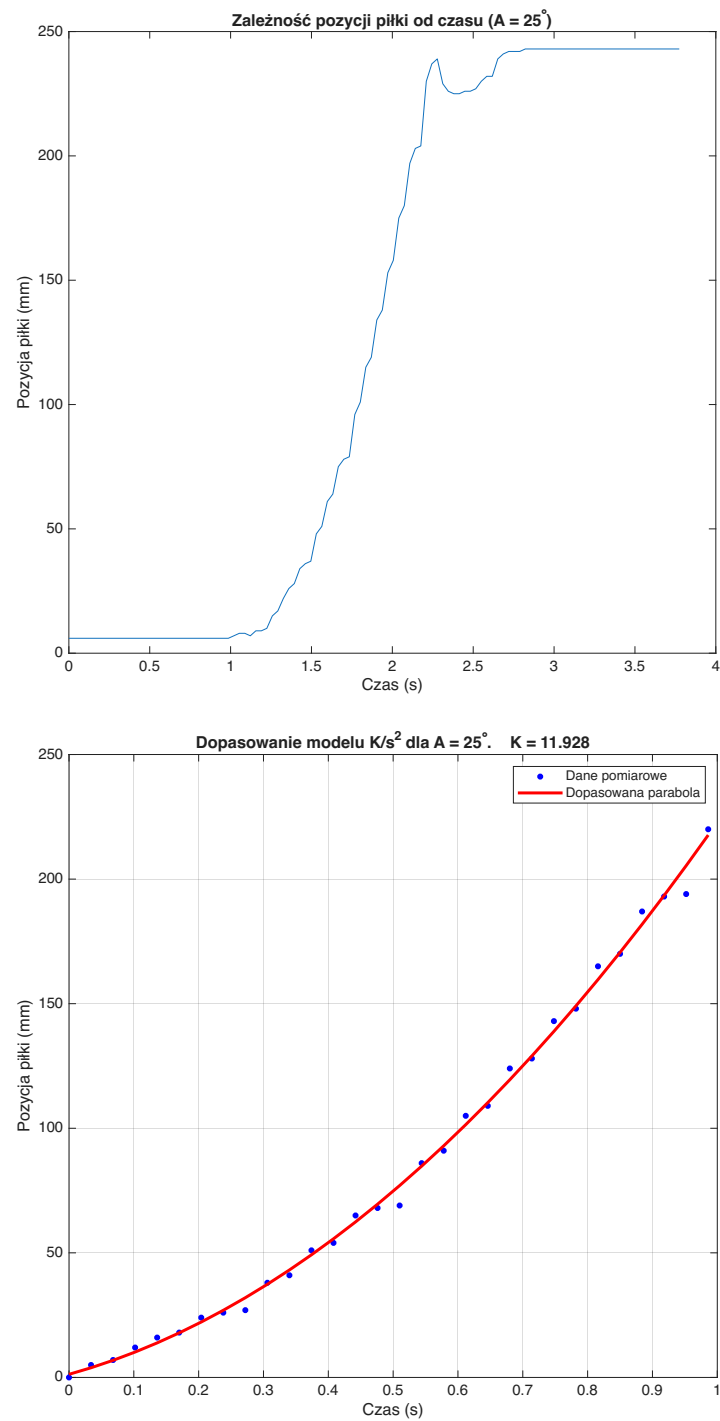
Rysunek 3.4: Wyniki identyfikacji dla pozycji zadanej 112° .



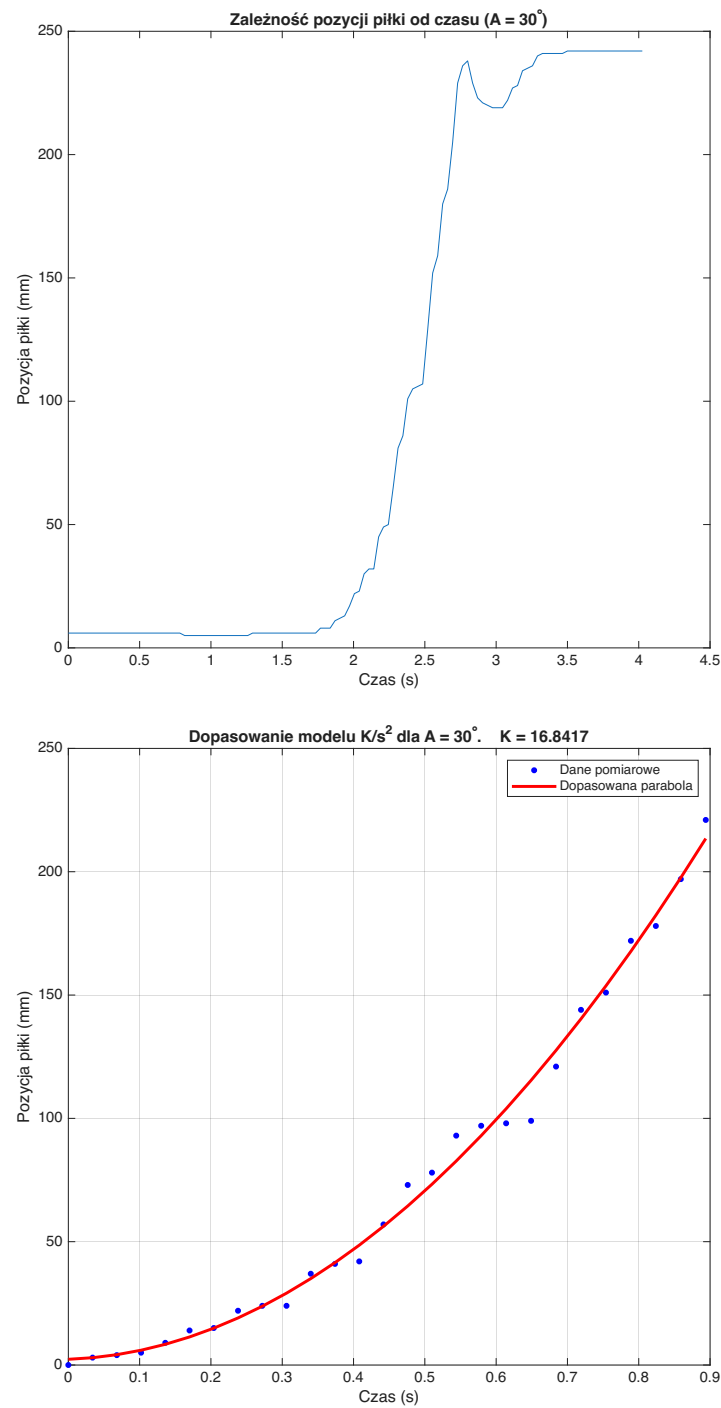
Rysunek 3.5: Wyniki identyfikacji dla pozycji zadanej 115° .



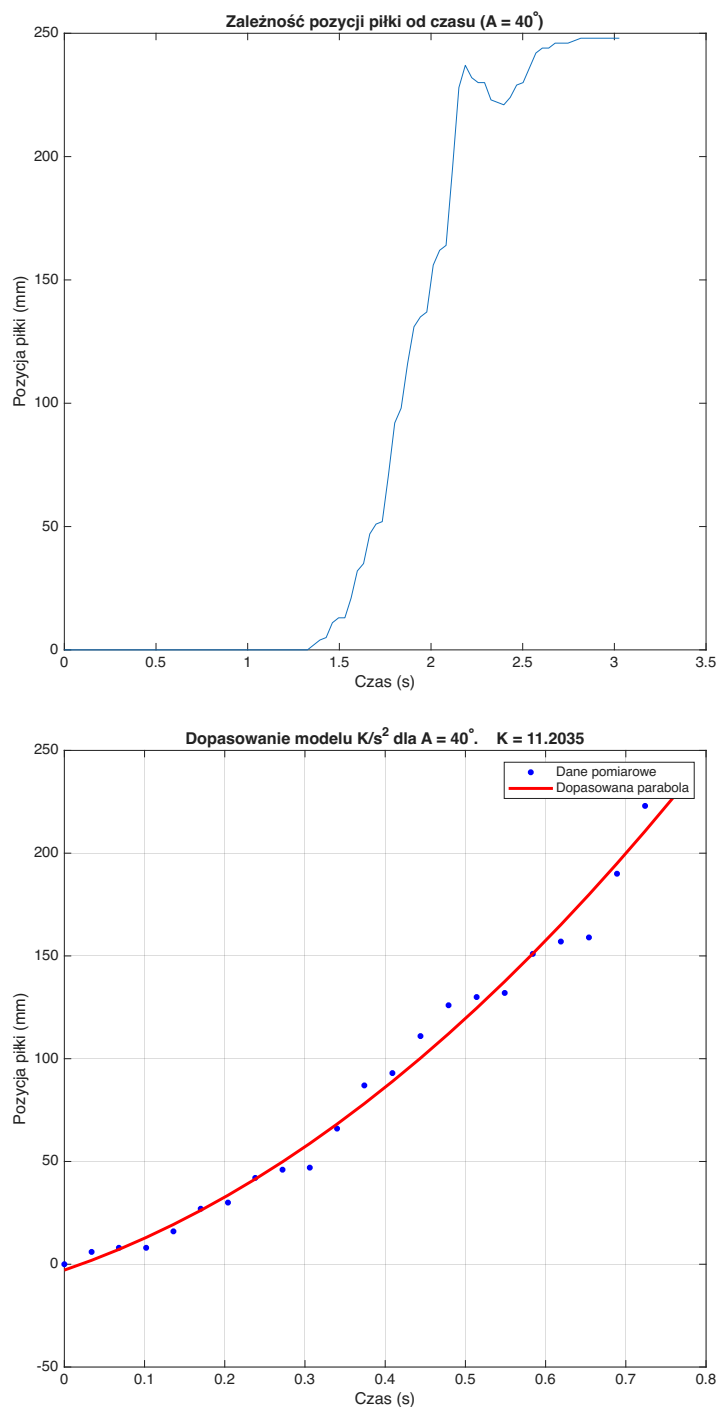
Rysunek 3.6: Wyniki identyfikacji dla pozycji zadanej 120° .



Rysunek 3.7: Wyniki identyfikacji dla pozycji zadanej 125° .



Rysunek 3.8: Wyniki dla pozycji zadanej 130° – próba odrzucona z uwagi na widoczny wpływ dynamiki serwomechanizmu.



Rysunek 3.9: Wyniki dla pozycji zadanej 140° – próba odrzucona z uwagi na widoczny wpływ dynamiki serwomechanizmu.

3.2.2 Porównanie wzmocnienia teoretycznego i doświadczalnego

Wzmocnienie układu kulka–belka wyznaczono dwiema niezależnymi metodami. Przed zestawieniem wyników konieczne jest ujednolicenie jednostek, ponieważ obie metody

operują na różnych układach pomiarowych.

Wzmocnienie teoretyczne. Na podstawie wyprowadzonego modelu wzmocnienie statyczne układu wynosi:

$$K_{\text{teor}} = k_{\text{mech}} \cdot c_{\text{ball}} = 0,20 \times 5,886 = 1,177 \frac{\text{m}}{\text{s}^2 \cdot \text{rad}}, \quad (3.20)$$

gdzie wejście u wyrażone jest w radianach, a wyjście x w metrach.

Wzmocnienie doświadczalne – przeliczenie jednostek. Estymacja metodą odpowiedzi skokowej dała wynik w jednostkach pomiarowych eksperymentu:

$$K_{\text{exp}} = 11,9318 \frac{\text{mm}}{\text{s}^2 \cdot ^\circ}. \quad (3.21)$$

Przeliczenie na układ SI wymaga przeliczenia milimetrów na metry oraz stopni na radiany:

$$K_{\text{exp,SI}} = K_{\text{exp}} \cdot \frac{10^{-3} \text{ m/mm}}{\pi/180 \text{ rad/}^\circ} = 11,9318 \times \frac{10^{-3}}{0,017453} \approx 0,684 \frac{\text{m}}{\text{s}^2 \cdot \text{rad}}. \quad (3.22)$$

Porównanie. Zestawienie obu wartości w tych samych jednostkach przedstawiono w tabeli 3.1.

Tabela 3.1: Porównanie wzmocnienia teoretycznego i doświadczalnego

Metoda	Wartość $\left[\frac{\text{m}}{\text{s}^2 \cdot \text{rad}} \right]$	Źródło
Teoretyczna ($k_{\text{mech}} \cdot c_{\text{ball}}$)	1,177	wyprowadzenie analityczne
Doświadczalna (odpowiedź skokowa)	0,684	identyfikacja eksperymentalna
Stosunek $K_{\text{teor}}/K_{\text{exp,SI}}$	1,72	—

Różnica rzędu 72% wynika z uproszczonego modelu przełożenia mechanicznego oraz założenia idealnego toczenia kulki bez poślizgu. Do strojenia regulatorów przyjęto wartość doświadczalną jako bliższą rzeczywistej dynamice obiektu.

3.2.3 Przyjęte parametry modelu

Tabela 3.2: Parametry numeryczne modelu przyjęte do syntezy regulatorów

Parametr	Wartość	Jednostka	Źródło
T_{serwo}	0,095	s	karta katalogowa, interpolacja
c_{ball}	5,886	$\frac{\text{m}}{\text{s}^2 \cdot \text{rad}}$	wyprowadzenie analityczne
k_{mech}	0,20	—	pomiar geometryczny (3 cm/15 cm)
K_{exp}	11,9318	$\frac{\text{mm}}{\text{s}^2 \cdot ^\circ}$	identyfikacja doświadczalna
$K_{\text{exp,SI}}$	0,684	$\frac{\text{m}}{\text{s}^2 \cdot \text{rad}}$	przeliczenie z K_{exp}

3.3 Projekt regulatora LQR

3.3.1 Opis przestrzeni stanów

Jako wektor stanu przyjęto trzy fizycznie mierzalne wielkości:

$$\mathbf{x} = \begin{bmatrix} x \\ \dot{x} \\ \theta \end{bmatrix}, \quad (3.23)$$

gdzie x — pozycja kulki, $\dot{x}$ — prędkość kulki, θ — kąt belki.

Wyprowadzając równania różniczkowe z bloków transmitancyjnych, otrzymuje się model przestrzeni stanów w postaci $\dot{\mathbf{x}} = A\mathbf{x} + Bu$, $y = C\mathbf{x}$:

$$\begin{bmatrix} \dot{x}_1 \\ \dot{x}_2 \\ \dot{x}_3 \end{bmatrix} = \begin{bmatrix} 0 & 1 & 0 \\ 0 & 0 & c_{\text{ball}} \\ 0 & 0 & -1/T_{\text{serwo}} \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} + \begin{bmatrix} 0 \\ 0 \\ k_{\text{mech}}/T_{\text{serwo}} \end{bmatrix} u, \quad (3.24)$$

$$y = \begin{bmatrix} 1 & 0 & 0 \end{bmatrix} \mathbf{x}. \quad (3.25)$$

Podstawiając wartości liczbowe ($c_{\text{ball}} = 5,886$, $T_{\text{serwo}} = 0,095$):

$$A = \begin{bmatrix} 0 & 1 & 0 \\ 0 & 0 & 5,886 \\ 0 & 0 & -10,526 \end{bmatrix}, \quad B = \begin{bmatrix} 0 \\ 0 \\ 10,526 \end{bmatrix}, \quad C = \begin{bmatrix} 1 & 0 & 0 \end{bmatrix}. \quad (3.26)$$

3.3.2 Sterowalność

Macierz sterowalności $\mathcal{C} = [B \ AB \ A^2B]$:

$$\mathcal{C} = \begin{bmatrix} 0 & 0 & 12,389 \\ 0 & 12,389 & -110,796 \\ 10,526 & -110,796 & 1,166 \cdot 10^3 \end{bmatrix}. \quad (3.27)$$

Wyznacznik:

$$\det(\mathcal{C}) = 12,389 \times (-130,380) \approx -1614 \neq 0. \quad (3.28)$$

Macierz ma pełny rząd 3, zatem układ jest **w pełni sterowalny** — możliwe jest zaprojektowanie efektywnego regulatora stanu (LQR).

3.3.3 Obserwowalność

Macierz obserwowalności $\mathcal{O} = \begin{bmatrix} C \\ CA \\ CA^2 \end{bmatrix}$

$$\mathcal{O} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1,177 \end{bmatrix}, \quad \det(\mathcal{O}) = 1,177 \neq 0. \quad (3.29)$$

Rząd wynosi 3, układ jest **w pełni obserwowalny** — wszystkie stany można zrekonstruować z pomiaru pozycji kulki x .

3.3.4 Regulator LQR

Regulator LQR (*Linear Quadratic Regulator*) minimalizuje wskaźnik jakości kwadratowej:

$$J = \int_0^\infty (\mathbf{e}^T Q \mathbf{e} + u^T R u) dt, \quad (3.30)$$

gdzie $\mathbf{e} = \mathbf{x} - \mathbf{x}_{\text{ref}}$ jest wektorem uchybu stanu, a $\mathbf{x}_{\text{ref}} = [r, 0, 0]^T$ odpowiada zadanej pozycji kulki r , zerowej prędkości i zerowemu kątowi belki.

TODO dodać jakieś przykładowe macierze Q i R / skrypt w pythonie gdzie to liczyliśmy musi to być w tym miejscu

Uzyskany wektor wzmocnień $\mathbf{K}$ przeniesiono bezpośrednio do modułu `lqr.c` firmware. Sygnał sterujący wyznaczany jest jako:

$$u = -\mathbf{K} \mathbf{e} = -[K_1 \ K_2 \ K_3] \begin{bmatrix} x - r \\ \dot{x} \\ \theta \end{bmatrix}. \quad (3.31)$$

3.4 Projekt regulatora PID

W firmware mikrokontrolera zaimplementowano klasyczny regulator PID z następującymi modyfikacjami:

- **pochodna na pomiarze** — różniczkowanie sygnału wyjścia (nie uchybu), co eliminuje skok pochodnej przy zmianie wartości zadanej;

- **anti-windup** (ograniczenie członu całkującego) — zapobiega nasyceniu członu I przy wysyceniu wyjścia;
- **dyskretyzacja** — metoda prostokątów (Euler przód) z krokiem $T_s = 30$ ms.

TODO uzasadnić dlaczego pochodna na pomiarze i dokończyć ten rozdział task @Banaszak !!!!!!!

Nastawy regulatora PID (K_p , K_i , K_d) dobierane są iteracyjnie i przesyłane do mikrokontrolera w czasie rzeczywistym przez aplikację desktopową bez konieczności ponownego programowania układu.

Wyniki eksperymentalne

TODO task @Jan !!!! - akwizycja z stanowiska - simulink z danymi z stanowiska i generowaniem odpowiedzi z modelu -opis

4.1 Metodyka przeprowadzonych badań

4.2 Wyniki pomiarów

4.3 Analiza i omówienie wyników

Bibliografía

- [1] Carlos G. Bolívar-Vincenty and Gerson Beauchamp-Báez. Modelling the ball-and-beam system from newtonian mechanics and from lagrange methods. In *Proceedings of the 12th Latin American and Caribbean Conference for Engineering and Technology (LACCEI'2014)*, Guayaquil, Ecuador, July 2014. Paper #PE148.
- [2] STMicroelectronics. *STM32F767ZI Reference Manual (RM0410)*, 2020. Rev 4.
- [3] STMicroelectronics. *VL53L0X World's Smallest Time-of-Flight Ranging and Gesture Detection Sensor*, 2018. Datasheet Rev 1.1.
- [4] Pushkar Kadam, Gu Fang, and Ju Jia Zou. Object tracking using computer vision: A review. *Computers*, 13(6):136, 2024.
- [5] A. Taifour Ali, A. M. Ahmed, H. A. Almahdi, Osama A. Taha, and A. Naser aldeen. Design and implementation of ball and beam system using PID controller. *Automatic Control and Information Sciences*, 3(1):1–4, 2017.
- [6] Katsuhiko Ogata. *Modern Control Engineering*. Prentice Hall, Upper Saddle River, NJ, 5 edition, 2010.
- [7] Gene F. Franklin, J. David Powell, and Abbas Emami-Naeini. *Feedback Control of Dynamic Systems*. Pearson, 7 edition, 2014.

Spis rysunków

2.1	Stanowisko laboratoryjne układu kulka na belce	6
2.2	Schemat blokowy stanowiska: połączenia podsystemów i przepływ sygnałów (elektrycznych, optycznych i mechanicznych) między aplikacją PC, mikrokontrolerem STM32, sensorami, serwomechanizmem oraz obiektem regulacji	7
2.3	Płytką ewaluacyjną STM32 Nucleo-F767ZI	8
2.4	Serwomechanek TowerPro MG946R	8
2.5	Kamera USB używana w systemie wizyjnym	9
2.6	Okno aplikacji desktopowej (PySide6)	11
2.7	Widok zakładki detekcji kulki	14
2.8	Stanowisko z nałożonymi wykrytymi znacznikami ArUco: ID 2 (lewy koniec), ID 0 (prawy koniec), ID 1 i ID 3 (punkty referencyjne ramy)	16
3.1	Rozkład sił działających na kulkę na równi pochyłej	19
3.2	Schemat mechanicznego połączenia serwa z belką (dźwignia)	20
3.3	Wyniki identyfikacji dla pozycji zadanej 109°: surowe dane pozycji kulki (górze) oraz dopasowanie paraboliczne (dół).	23
3.4	Wyniki identyfikacji dla pozycji zadanej 112°.	24
3.5	Wyniki identyfikacji dla pozycji zadanej 115°.	25
3.6	Wyniki identyfikacji dla pozycji zadanej 120°.	26
3.7	Wyniki identyfikacji dla pozycji zadanej 125°.	27
3.8	Wyniki dla pozycji zadanej 130° – próba odrzucona z uwagi na widoczny wpływ dynamiki serwomechanizmu.	28
3.9	Wyniki dla pozycji zadanej 140° – próba odrzucona z uwagi na widoczny wpływ dynamiki serwomechanizmu.	29